1. SPEZIFIKATION
   - Abstrakte Datentypen
   - Header-Dateien (.h) *
2. STACKS & QUEUES
   - Array Implementierung *
   - Listen Implementierung *
3. VERERBUNG & TEMPLATES
   - Ableitungen & Typverträglichkeit
   - Polymorphie Datenstrukturen
   - Templates & Schablonen
4. LISTEN & DICTIONARIES
   - Einfach verkettete Listen *
   - Doppelt verkettete Listen
   - Dictionary & Word-Count *
5. Graphalgorithmen
   - Spezifikation Graphen
   - BFS & DFS *
   - Test auf azyklisch
   - Dijkstra & Priority Queues
   - Topologisches Sortieren
6. Netzwerkflussprobleme
   - Labeling Algorithmus
7. Algorithmische Geometrie
   - Konvexe Hüllen
   - Gift Wrapping
   - Graham-Scan
   - Orientierung Prädikat *

EA: Java vs. Cpp
- Copy-by-Value bzw. -Referenz
- Call-by-Value bzw. -Referenz

BESPRECHUNG
- Abstrakter Datentyp → Spezifikation mit .h
  - Operationen auf die Datentypen anschauen. (erwähnt, aber nicht genutzt)
  - Stack, point_list, knot, template_list schreiben. (Grundidee mit Übung)
  - Spezifikation, Implementierung Priority Queue
  - Berechnung konvexe Hülle mit Gift Wrap & Scan

1. SPEZIFIKATION

- Ein abstrakter Datentyp besteht aus einer Menge von Objekten (Werte, Zahlen, ...) oder Zuständen, sowie einer Menge von Operationen auf dieser Menge.

- Zur Definition eines ADT gehört: Name, Menge der Objekte (Zstd), Menge der Operationen (Syntax: Name, Ein- & Ausgabetypen, Semantik: eigentliche Funktion).

z.B. Name: Integer
Objekte: $\mathbb{Z}$
Operationen: $+: \mathbb{Z} \to \mathbb{Z}$ (Syntax), Addiert Zahlen (Semantik)
$-: \mathbb{Z} \to \mathbb{Z}$ (Syntax), Subtrahiert Zahlen (Semantik)
...

1.1 Header-Dateien

- In Cpp werden Spezifikation und Implementierung getrennt in Header-Dateien (.h) und Code-Dateien (.cpp). Die Header Datei wird mit #include "xmpl.h" eingebunden.

  => Mehrere Implementierungen pro Spezifikation möglich

xmpl.h (SPEZIFIKATION)

! Spezifikation auf Kommentare achten!

```
class xmpl {

protected:

    string str;          // Speichert Namen

public:

    xmpl (string name);  // Konstruktor
    ~xmpl ();            // Destruktor

    void say_hello();    // Hello xyz ausgeben
```

xmpl.cpp (IMPLEMENTIERUNG)

```
#include "xmpl.h"

xmpl:: xmpl(string name){
    str = name;
}

xmpl:: say_hello() {
    cout << "Hello " << str << "." << endl;
}
```

Cheat-Sheet: Cpp Pointer

new: liefert Pointer auf den Stack!

```
int x = 5;
int* p = &x;    // & liefert Adresse von x.

*p == x;  // *p dereferenziert Pointer
*p = 7    // schreibt über den Pointer

int* a = new int;          // einzelnes Objekt init
int* A = new int[AOJ];     // Array init

void f(obj o)   // Call by value
void f(obj& o)  // Call by reference
void f(obj* o)  // Call by pointer

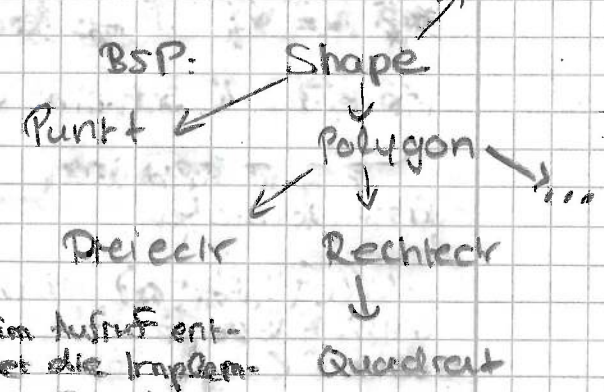
Obj o = new Obj();
(*o).p = 2      // Dereferenzieren, dann Zugriff
o->p = 2        // Zugriff über den Pointer.
```

2. VERERBUNG & TEMPLATES

- Es wird ein Datentyp benötigt, der sehr ähnlich zu einem anderen ist, d.h: (Klassen und Vererbung)

1) Einen Teil der Daten/Operationen wiederverwenden
2) Ander hinzufügen
3) Andere modifizieren

BSP:



Shapes.h

```
class Polygon {
private:
    ...
protected:
    ...
public:
    ...
    virtual double area();          // Fläche
    virtual double perimeter();     // Umfang
    virtual void translate(double dx, double dy);   // Verschieben
    ...
};
```

? VIRTUAL sagt: Beim Aufruf entscheidet Objekt über die Implementierung, nicht der Zeigertyp

VERERBUNG

```
class Rechteck : public Polygon
private:
    ...
protected:
    ...
public:
    ...
    double area() {                 // ÜBERSCHRIEBEN
        return w*h;
    }
    double perimeter() {
        return 2*(w+h);
    }
    ...
};
```

Bei kleinen Funktionen ist Aufwand für Aufruf+Rückkehr oft zu hoch.
⇒ Kann im Header inline stehen.

WICHTIG: Polymorphie klappt in CPP nur über Pointer, nicht über Objekt-Werte!

2.1 Typverträglichkeit (Polymorphie)

- Nun ist jede Unterklasse mit ihrer Oberklasse verträglich. Achtung: Dies gilt nicht andersherum

```
Polygon* p = new Rechteck();        ✓
Rechteck* r = new Polygon();        ✗
```

Arrays von Pointern !

```
Polygon** A = new Polygon*[100];    ✓
A[0] = new Rechteck(..)

Rechteck** A = new Rechteck*[100];  ✗
A[0] = new Polygon(..)
```

- Spezifikation und Implementierung von picture, um mehrere shapes zu verwalten.

```
class shape {                          ohne die würde Shape Implementierung verwendet werden!
public:
    virtual void translate(double dx, double dy);
    virtual void rotate(double x, double y);
    virtual void write(ostream& out);
    virtual void read(istream& in);
    virtual void draw(window& win);
};
class picture {
protected:
    shape * A[100];   // Polymorpher Container
    int count;
public:
    picture() {count = 0;}
    int insert(shape* s) {
        A[count] = s;
        count++;
        return count - 1;
    }
    void remove(int i) {
        A[i] = A[count];
        count--;
    }
    void draw(window& win) {
        for(int i=0; i<count; i++) {
            A[i]->draw(win);
        }
    }
    ... ähnl. zu draw
};
```

2.2 Templates

- Eine generische DS ist bspw. eine Liste von beliebigen Objekten. Um diese wiederzuverwenden braucht man Templates.

```
template <class T>
void swap(T& a, T& b){                // Vertauscht zwei Instanzen einer beliebigen Klasse
    T tmp = a;
    a = b;
    b = tmp;
}
```

Wäre auch mit mehreren machbar:
template<class A, class B>

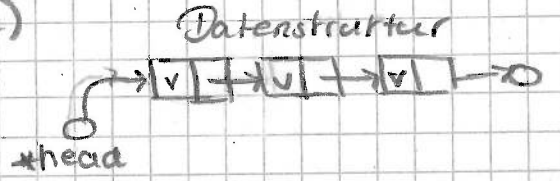
Nutzung:

```
int x = 1;
int y = 2;
swap<int>(x,y);
```

Template muss explizit aufgelöst werden!

3. LISTEN & DICTIONARIES

3.1 Einfach verkettete Liste (Generisches Template)



list.h

```
template<class T>
class list_elem{
public:
    list_elem* next;
    T value;

    list_elem(list_elem* next, T value){
        this->next = next;
        this->value = value;
    }
    ~list_elem(){} <- wenn leerer Destruktor, dann gar nicht notwendig.
};

template<class T>
class list{
protected:
    list_elem<T>* head;
public:
    list(){
        head=NULL;
    }
    void push(T value){
        list_elem<T> elem = new list_elem<T>(head, value);
        head = elem;
    }
    T pop(){
        if(head == NULL){
            return NULL
        }
        T value = head->value;
        head = head->next;
        return value;
    }
};
```

main.cpp ← Beispielhafte Anwendung

```
list<point> points = list<point>()
points.push(Point(1,2));
point p = points.pop();
```

3.2 Einfach verkettete Listen (durch Vererbung)

gen_list.h

```
class list_elem {
public:
    list_elem * next;

    list_elem (list_elem* next) {
        this->next = next;
    }
};

class list {
protected:
    list_elem* head;

public:
    list() {
        head = NULL;
    }
    void push (list_elem* elem) {
        elem->next = head;
        head = elem;
    }
    list_elem* pop() {
        if (head == NULL) {
            return NULL;
        }
        list_elem* elem = head;
        head = head->next;
        return elem;
    }
};
```

→ Copy Constructor

```
list (list &other) { }
```

point.cpp

```
class point : public list_elem {
public:
    int x, y;
    point (int x, int y) {
        this->x = x;
        this->y = y;
    }

    int comp_xy(point& other) {
        if (x < other.x) return -1;
        if (x > other.x) return 1;
        if (y < other.y) return -1;
        if (y > other.y) return 1;
        return 0;
    }
};
```

$$p.comp_xy(q) = \begin{cases} -1, & p <_{xy} q \\ 0, & p =_{xy} q \\ 1, & p >_{xy} q \end{cases}$$

```
class point_list : public list {
public:
    void push(point* p) {
        list::push(p);
    }
    point* pop() {
        return (point*) list::pop();
    }
};
```

Überschriebener Interface erlaubt nun noch Punkte, durch Polymorphie Pointer verträglich mit Oberklasse.

↑ kann sicher gecastet werden.

Template List (Container)

```
template<class T>
class container: public list_elem {
public:
    T data;
};

template<class T>
class container_list : list {
public:
    void push(T* t) {
        list::push(t);
    }
    container pop() {
        return (container*) list::pop();
    }
};
```

3.3 Dictionaries (Generisch) ★

- Eingabe ist eine Folge von Strings. Zähle das Vorkommen von jedem der Wörter. (Word-Count-Problem)

dictionary.h

```
template<class K, class V>
class dict{
public:
    void insert(K, V);
    bool lookup(K) const;
    V translate(K) const;
    void remove(K);
    void update(K, V);
};
```

const markiert, dass Zustand/Felder des DS nicht verändert wird.

Nicht dringend notwendig, hilft hier aber!

word-count.cpp

```
dict<string, int> count;
string word;
while (cin >> word) {
    if (count.lookup(word)) {
        int c = count.translate(word) + 1;
        count.update(word, c);
    }
    else{
        count.insert(word, 1);
    }
}
```

3.4 Priority Queues

p_queue.h

Info ← Priorität

```
template<class I, class P>
class p_queue{
public:
    pq_item insert(I info, P prio);
    pq_item del_min();
    pq_item decrease_p(pq_item, item);
};
```

Wiederholungen in p_queue erlaubt!
⇒ man braucht item um Eintrag zu finden.

3.4 Priority Queues

```
template<class P, class I>
class pq_item {
public:
    P prio;
    I info;

    ...
};

template<class P, class I>
class pq {
public:
    // Füge neues Item ein, ordne in Queue gemäß Prio ein. Gibe
       pq_elem aus, welches auf Stelle in Queue zeigt
    pq_item insert (P prio, I info);

    void set_prio (pq_item index, P prio);
    // Updatet prio eines Elements in Queue. (Womöglich
       flexible als decrease/increase. Aktualisiert die Queue.

    void del_min ();
    // Entfernt das Minimum (Priorität).

    pq_item find_min ();
    // Gibt Item des aktuellen Minimums aus.

    I info (pq_item item); // Gibt Prio/Info des Items aus.
    P prio (pq_item item);
};
```

4. STACKS & QUEUES

- Stacks mit Listen oder Arrays implementieren. (FILO Prinzip)

stack_array.h

```
template <class T>
class stack {
protected:
    T* field;
    int cap, head;
public:
    stack (int cap) {
        this->cap = cap;
        this->top = 0;
        field = new T[cap];
    }
    void push (T& t) {
        if (this->full()) {
            EXCEPTION("Stack ist voll");
        }
        field[top] = t;
        top++;
    }
    bool full() {
        return top == cap;
    }
    bool empty() {
        return top == 0;
    }
    T top() {
        if (this->empty()) {
            EXCEPTION("Stack ist leer");
        }
        return field[top-1];
    }
    T pop() {
        T t = this->top();
        top--;
        return t;
    }
};
```

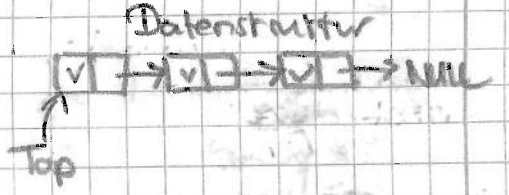
Field ist Feld von T, wenn ** wäre es ein Feld von Pointern.

Cpp Exception

Helfer Methoden

stack_ptr.h

```
template<class T>;
class stack_elem{
public:
    T value;
    stack_elem* prev;
    stack_elem(T value){
        this->value = value;
        this->prev = NULL;
    }
}
```




```
template<class T>
class stack{
protected:
    stack_elem<T>* top;
public:
    stack(){
        top = NULL;
    }
    void push(T value){
        stack_elem<T>* elem = new stack_elem<T>(value);
        elem->prev = top;
        top = elem;
    }
    bool empty(){
        return top == null;
    }
    T top(){
        if(empty()){
            EXCEPTION("Stack ist leer");
        }
        return top->value;
    }
    T pop(){
        T t = top();
        top = top->prev;
        return t;
    }
}
```

PRO: Keine Größenbeschränkung
CON: 2 Klassen, weniger Performance

- Queues lassen sich mit Listen oder Arrays implementieren.

queue_array.h

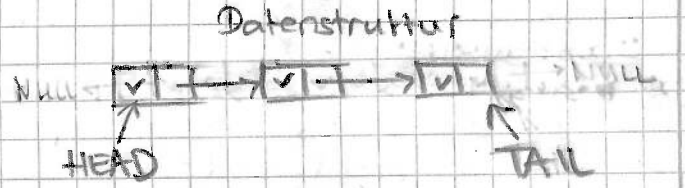
```
template <class T>
class queue {
protected:
    T* field;
    int cap, head, tail;
public:
    queue (int cap) {
        field = new T[cap];
        this -> cap = cap;
        head = 0;
        tail = 0;
    }
    bool empty () {
        return head == tail;
    }
    bool full () {
        return ((head+1) % cap == tail;
    }
    void enqueue(T value) {
        if (full()) {
            EXCEPTION("Queue ist voll");
        }
        field[head] = value;
        head = (head+1) % cap;
    }
    T dequeue () {
        if (empty()) {
            EXCEPTION("Queue ist leer");
        }
        T t = field[tail];
        tail = (tail+1) % cap;
        return t;
    }
}
```

queue_ptr.h

```
template<class T>
class elem {
public:
    elem* next;
    T value;
};

template<class T>
class queue {
protected:
    elem<T> head;
    elem<T> tail;
public:
    queue(){
        head = NULL;
        tail = NULL;
    }
    bool empty(){
        return head == NULL;
    }
    void enqueue(T value){
        elem<T> el = new elem<T>(value);
        if(!empty()){ tail->next = el; }
        tail = el;
    }
    T dequeue(){
        if(empty()){
            EXCEPTION("Queue ist leer");
        }
        T t = head->value;
        head = head->next;
        return t;
    }
};
```

Datenstruktur




```
elem(T value){
    this->value = value;
}
```

5. GRAPHEN

- LEDA nutzt den allg. Graph Datentyp graph:

```
class graph {
public:                                  bool empty();
    node source(edge e);
    node target(edge e);
    int outdeg(node v);                  } Datenzugriff
    int indeg(node v);
    list<edge> out_edges(node v);

    node new_node();
    edge new_edge(node v, node w);       } Datenmanipulation
    void del_node(node v);
    void del_edge(edge e);
    ------------------------------------
    forall_nodes(v, G) {...}
    forall_edges(e, G) {...}             } Iteratoren
    forall_out_edges(e, v) {...}
    forall_in_edges(e, v) {...}
```

node_array<T> A(G,val) — Initialisiert Array der Größe von #Knoten
edge_array<T> B(G,val) — Initialisiert Array der Größe von #Kanten

* wird in diesem Param bereitgestellt!

5.1 BFS ausführen ★ Queue → BFS, Stack → DFS

- Gebe alle von einem Startknoten s erreichbaren Knoten aus

```
list<node> dfs(graph g, node s) {
    list<node> reachable;
    queue<node> queue;
    node_array<bool> visited = node_array<bool>(g,false);

    queue.enqueue(s);
    while(!q.empty()){
        node v = queue.dequeue();
        edge e;

        forall_out_edges(e,v){
            node u = G.target(e);
            if(!visited[u]){
                reachable.push(u);
                queue.enqueue(u);
                visited[u]=true;
            }
        }
    return reachable;
}
```

Initialisiert Array der Größe von #Knoten.

BFS: Bei Blickkontakt markieren.

MERKSATZ / ESELSBRÜCKE !

BFS: Bei Blickkontakt markieren
DFS: Bei Durchlauf markieren.

5.2 DFS durchführen

```
list<node> dfs(graph g, node s){
    list<node> reached;
    node_array<bool> visited = node_array<bool>(g, false);
    stack<node> todo;

    todo.push(s);
    while(!todo.empty()){
        node v = todo.pop();
        if(visited[v]){
            continue;
        }
        visited[v]=true;
        reachable.push(v);
        edge e;
        forall_out_edges(e,v){
            node u = g.target(e);
            todo.push(u);
        }
    }
    return reachable;
}
```

DFS: Bei Durchlauf markieren

5.3 Azyklische Graphen

- Test ob ein Graph azyklisch ist oder eben nicht.
- Entferne Knoten mit Eingangsgrad 0 so lange, bis nicht mehr geht. Ist Graph am Ende leer, dann ist er azyklisch.

```
bool is_acyclic(graph g){
    stack<node> ZERO;
    node v;
    forall_nodes(v,g){
        if(g.indeg(v)==0){
            ZERO.push(v);
        }
    }

    while(!ZERO.empty()){
        node v=ZERO.pop();
        edge e;
        forall_out_edge(e,v){
            node u = g.target(e);
            g.del_edge(e);
            if(g.indeg(u)==0){
                ZERO.push(u);
            }
        }
        g.del_node(v);
    }
    return g.empty();
}
```

5.4 Dijkstra Algorithmus

```
node_array<int> dijkstra(graph& g, edge_array<int>& cost, node s){
    node_array<int> dist = node_array<int>(g, numeric_limits<int>::max())
    p_queue<node, int> queue;                                  // priority Queue
    node_array<pq_item> indices(g, NULL);                      // wo ist node in der queue?
    dist[s] = 0;                                               // gesetzt 2^31 - 1.
    queue.insert(s, 0);
    while(! queue.empty()){
        node v = queue.del_min();
        edge e;
        forall_out_edge(e, v){
            node u = g.target(e);
            int d = dist[v] + cost[e];
            if (d < dist[u]){
                if(dist[u] == numeric_limits<int>::max()){
                    //u erstes mal besucht, muss in Queue
                    indices[u] = queue.insert(u, d);
                }
                else {
                    // u in Queue, gewicht updaten.
                    queue.decrease_p(indices[u], d);
                }
                dist[u] = d;
            }
        }
    }
    return dist;
}
```

ohne die können wir die priorities nicht mehr updaten.

5.5 Topologisches Sortieren

```
list<node> topsort(graph& g){
    node_array<int> indeg = node_array<int>(g);
    list<node> zero;
    node v;
    forall_nodes(v, g){
        indeg[v] = g.indeg(v);          // Simulieren löschen in Array, um Graphen nicht zu verändern.
        if(indeg[v] == 0){
            zero.append(v);
        }
    }
    list<node> topo;
    while(!zero.empty()){
        node v = zero.pop();
        topo.append(v);
        edge e;
        forall_out_edges(e, v){
            node u = g.target(e);
            indeg[u]--;
            if(indeg[u] == 0){
                zero.append(u);
            }
        }
    }
    return topo;
}
```

Kann man genauso mit nur einer Queue lösen.

ALTERNATIV:
- topo array referenziert mit rein geben und bool ausgeben. (in Liste packen)

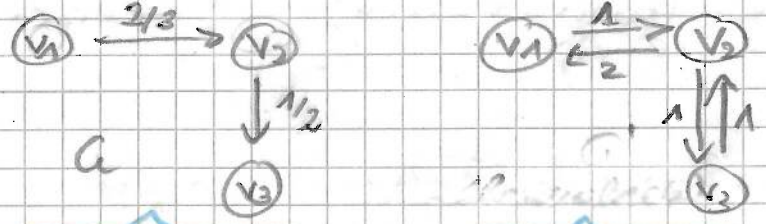
⇒ Kann ausgeben ob erfolgreich oder nicht.

5 FLUSSNETZWERKE ($G=(V,E), S \subseteq V, T \subseteq V, c: E \to \mathbb{R}_0^+$)

Quellen — Senken — Kapazitäten

Finde $f: E \to \mathbb{R}_0^+$, s.d.:

- $\forall e \in E: 0 \leq f(e) \leq c(e)$ (Kapazitätseinhaltung)
- $\forall v \in V \setminus S \cup T: \sum_{(v,v) \in E} c(v,v) = \sum_{(v,w) \in E} c(v,w)$ (Flusserhaltung)



Flussnetzwerk — Residualnetzwerk

Vorwärts: $c(e) - f(e)$ „was noch rein passt"

Rückwärts: $f(e)$ „was noch weg kann"

- Max-Flow-Min-Cut-Theorem: Die Kapazität des größten Flusses ist so groß wie die des kleinsten (S,T)-Schnitts. (Flaschenhals)

ZIEL: Berechne Max-Flow ohne ein Residualnetz explizit als Graph zu definieren (zu teuer).

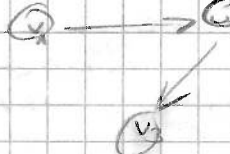
enthält genutzte Kanten gemäß f!

```
void augment(graphM g, node s, node t, node_array<edge>& pred,
             edge_array<int>& cap, edge_array<int>& flow){
  int delta = MAXINT;
  node v=t;
  while (v != s) {
    int r;
    edge e = pred[v];
    if (v == g.source(e)) {   // DFS nutzt Rückwärtskante gegangen
      r=flow[e];
      v=g.target(e);
    }
    else {                    // DFS nutzt Vorwärtskante.
      r = cap[e]- flow[e];
      v=g.source(e);
    }
    if (r < delta) {          // Fluss kann erhöht werden.
      delta = r;
    }
  }
```

Findet Flaschenhals des Restnetz. ⇒ kann erhöht werden.

- Augment erhöht den Fluss entlang der durch die BFS Pfades.

...



```
v=t;
while(v != s) {
    edge e = pred[v];
    if( v == g.source(e)){
        flow[e] = flow[e] - delta;
        v = g.target(e);
    }
    else {
        flow[e] = flow[e] + delta;
        v = g.source(e);
    }
}
```

// DFS nimmt Rückwärts ⇒ nehme Kap.

// DFS nimmt Vorwärts ⇒ erhöhe Kap.

Im Prinzip Ford Fulkerson.

```
edge_array<int> mf_labelling(graph& g, node s, node t, edge_array<int>& cap){

    edge_array<int> flow = edge_array<int>(g,0);
    node_array<bool> labeled = node_array<bool>(g,false);
    node_array<edge> pred = node_array<edge>(g,null);

    while(true) {
        queue<node> Q;
        labeled[s] = true;
        Q.append(s);

        while(!Q.empty()) {
            node v = Q.pop();

            edge e;
            forall_out_edges(e,v) {
                if(flow[e] == cap[e]) {      // Kapazität voll.
                    continue;
                }

                node u = g.target(e);        // Node bereits in Queue
                if(labeled[u]) {
                    continue;
                }

BFS ⇒ Blickrichtung {   labeled[u] = true;
                        pred[u] = e;
                        Q.append(u);
                    }

                forall_in_edges(e,v) {       // keinen Fluss auf e
                    if(flow[e] == 0) {
                        continue;
                    }

                    node u = g.source(e);    // Node bereits in Queue
                    if(labeled[u]) {
                        continue;
                    }
```

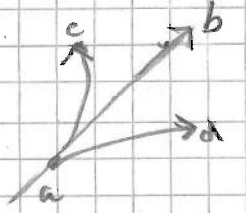
Führt BFS auf dem Residualnetz aus.

```
            labeled[u] = true;
            pred[u] = v;
            Q.append(u);
        }
        if (labeled[t]) {   // DFS hat Senke erreicht
            Q.clear();
        }
    }                                   => Denn noch s-t-Pfad in Restnetz
    if (labeled[t]) {       // Fluss kann noch erhöht werden
        augment(g, s, t, pred, cap, flow);
    }
    else {      // Fluss kann nicht mehr erhöht werden,
        break;
    }
}
return flow;
}
```

6. ALGO GEOMETRIE

$orient: \mathbb{Z} \times \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$

$$orient(a,b,c) = \begin{cases} 1, & c \text{ links von } \overrightarrow{ab} \\ 0, & c \text{ auf } \overrightarrow{ab} \\ -1, & c \text{ rechts von } \overrightarrow{ab} \end{cases}$$



$$orient(a,b,c) = sign \begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix}$$

$a_x \cdot b_y + a_y \cdot c_x + b_x \cdot b_y - c_x \cdot b_y - c_y \cdot a_x - b_x \cdot a_y$

(Regel von Sarrus)

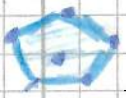
Quick Tipps:

1) $d(a,b) < d(x,y)$ ohne $\sqrt{}$ ausführen ⇒ bessere Performance und genauer!

2) Schnitt lässt sich mit orient einfach testen! ⇔ $\overline{ab}$ und $\overline{cd}$ schneiden



$orient(a,b,c) \neq orient(a,b,d) \wedge orient(c,d,a) \neq orient(c,d,b)$



ZIEL: Bestimme konvexe Hülle einer Punktemenge $P = \{p_1, \dots, p_n\}$.

```
list<point> gift_wrapping(list<point> P) {
    list<point> CH;
    point p = L.first();
    point q;
    forall(q, P) {                          // LEDA Funk
        if (p.cmp_xy(q) == -1) {            // } Findet den rechtesten Punkt
            p = q;
        }
    }
    CH.append(p);
    while(true) {
        point q = P.first();
        forall(p, P) {                      // } Finde den Punkt rechts daneben.
            if (CH.last().orient(q,p) == -1 || (CH.last().orient(q,p)
                == 0 && CH.last().cmp_dist(q,p) == -1)) {
                q = p;
            }
        }
        P.del(q);
        if (q == p) {
            break;
        }
        CH.append(q);
    }
    return CH;
}
```

$$cmp_dist(b,c) = \begin{cases} -1, & |\overline{ab}| < |\overline{ac}| \\ 0, & |\overline{ab}| = |\overline{ac}| \\ +1, & |\overline{ab}| > |\overline{ac}| \end{cases}$$

$$cmp_xy(a,b) = \begin{cases} -1, & a <_{xy} b \\ 0, & a =_{xy} b \\ 1, & a >_{xy} b \end{cases}$$

7. ITERATOREN

- Hilfskonzept um über Mengen zu iterieren. In der VL werden die + immer mit übergeben.

```
bool palindrom(Iterator a, Iterator b){
    b = b-1;
    while(a<b && (*a)==(*b)){          <- Entspricht die Pointer
        a = a+1;
        b = b-1;   // Einene Pointer
    }
    return a>=b;
}
```

BSP.:
```
int A[] = {1,2,3,4,5};
palindrom(A, A+5);
```

Merged Sortierte Folgen a..b und c..d.

```
template<class Iter>
void merge(Iter a, Iter b, Iter c, Iter d, Iter e){
    while(a!=b && c!=d){
        if(c==d || *a < *c){   // Hole aus a..b wenn a<c,
            e = *a;                oder c..d schon verbraucht.
            a++;
        }
        else{                  // Hole aus c..d
            e = *c;
            c++;
        }
        e++;
    }
}
```